

1. Single Inheritance

Explanation

Single inheritance means **one child class inherits properties and methods of one parent class**.

Real-life Example

- Parent: **Vehicle**
- Child: **Car**

A car *is a* vehicle.

Code Example

```
class Vehicle {
    void start() {
        System.out.println("Vehicle starts");
    }
}

class Car extends Vehicle {
    void drive() {
        System.out.println("Car is driving");
    }
}

public class SingleInheritance {
    public static void main(String[] args) {
        Car c = new Car();
        c.start();    // from Vehicle
        c.drive();    // from Car
    }
}
```

2. Multilevel Inheritance

Explanation

A class is derived from another derived class, forming a **chain of inheritance**.

Real-life Example

- Grandparent: **Animal**
- Parent: **Dog**
- Child: **Puppy**

Code Example

```
class Animal {
    void eat() {
        System.out.println("Animal eats");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}

class Puppy extends Dog {
    void play() {
        System.out.println("Puppy plays");
    }
}

public class MultilevelInheritance {
    public static void main(String[] args) {
        Puppy p = new Puppy();
        p.eat();
        p.bark();
        p.play();
    }
}
```

3. Hierarchical Inheritance

Explanation

Multiple child classes inherit from **one common parent class**.

Real-life Example

- Parent: **Account**
- Children: **SavingsAccount, CurrentAccount**

Code Example

```
class Account {
    void showAccount() {
        System.out.println("Account details");
    }
}

class SavingsAccount extends Account {
    void showSavings() {
        System.out.println("Savings Account");
    }
}
```

```

    }
}

class CurrentAccount extends Account {
    void showCurrent() {
        System.out.println("Current Account");
    }
}

public class HierarchicalInheritance {
    public static void main(String[] args) {
        SavingsAccount s = new SavingsAccount();
        CurrentAccount c = new CurrentAccount();

        s.showAccount();
        s.showSavings();

        c.showAccount();
        c.showCurrent();
    }
}

```

4. Multiple Inheritance (Using Interface)

Explanation

Java does **not allow multiple inheritance using classes**, but it allows it using **interfaces**. A class can implement **multiple interfaces**.

Real-life Example

- Person can be:
 - **Student**
 - **Employee**

Code Example

```

interface Student {
    void study();
}

interface Employee {
    void work();
}

class Person implements Student, Employee {
    public void study() {
        System.out.println("Person studies");
    }

    public void work() {

```

```

        System.out.println("Person works");
    }
}

public class MultipleInheritance {
    public static void main(String[] args) {
        Person p = new Person();
        p.study();
        p.work();
    }
}

```

5. Hybrid Inheritance (Using Interface)

Explanation

Hybrid inheritance is a **combination of two or more types of inheritance**. In Java, it is achieved using **interfaces**.

Real-life Example

- Device → Phone → Smartphone
- Smartphone also implements **Camera**

Code Example

```

interface Camera {
    void clickPhoto();
}

class Phone {
    void call() {
        System.out.println("Making a call");
    }
}

class Smartphone extends Phone implements Camera {
    public void clickPhoto() {
        System.out.println("Photo clicked");
    }
}

public class HybridInheritance {
    public static void main(String[] args) {
        Smartphone s = new Smartphone();
        s.call();
        s.clickPhoto();
    }
}

```